Contrôleur de feux de circulation

Initiation à Codesys

1. Présentation du sujet

Codesys est un environnement de développement intégré pour automates programmables industriels (API).

L'objectif est de s'initier à Codesys sur le thème des feux de circulation.

Nous ne disposons pas d'automate, nous utiliserons le PLC (programmable logic controller) simulé par le PC.

Ce travail s'étendra sur environ trois séances de quatre heures.



© Earthz Stocker

2. Premiers tests en langage LD, ST et SFC - un bouton, un feu, un timer

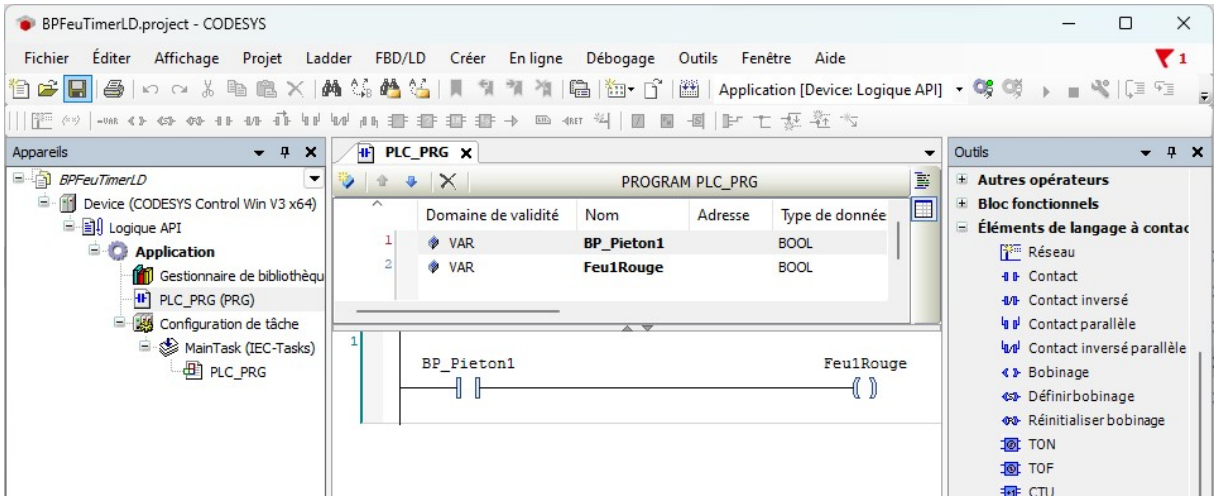
Lancer **Codesys V3.5 SP19 Patch3**.
Quand celui-ci a terminé de démarrer, cliquez dans le menu sur **Fichier**, **Nouveau projet**.
Dans la fenêtre qui apparaît, sélectionnez **Projet standard**, saisissez le nom du projet **BPFeuTimerLD**, et créez son dossier **H:\InitiationCodesys\BPFeuTimer**.

Dans la fenêtre qui apparaît alors, dans le champ **Appareil** choisissez **CODESYS Control Win V3 x64**, et dans le champ **PLC_PRG**, choisissez **Langage à Contacts(LD)**. Terminez par **OK**.

Ce que Codesys nomme **Appareil** est l'automate. En l'occurrence nous n'avons pas d'automate, nous le simulons avec le PC, ce qui nous simplifie la tâche pour une initiation.
Le **PLC_PRG** est le programme du Programmable Logic Controller, c'est-à-dire de l'automate.
On débute avec le langage à contacts, ou langage ladder.
C'est le langage qu'affectionnent les techniciens des entreprises de production, au sein du service de la maintenance. C'est un langage visuel, avec des interrupteurs pour les entrées, et des bobines pour les sorties, en références aux bobines d'entrées des relais des actionneurs.

2.1 Un bouton à bascule et un feu, en programmation ladder

Dans le volet de gauche **Appareils** se trouve l'arborescence des principaux objets du projet. Si nécessaire y double-cliquez sur **PLC_PRLG(PRG)** pour faire apparaître la fenêtre centrale **PLC_PRG** qui contiendra en partie haute les variables et en partie basse la programmation sous forme de réseaux.

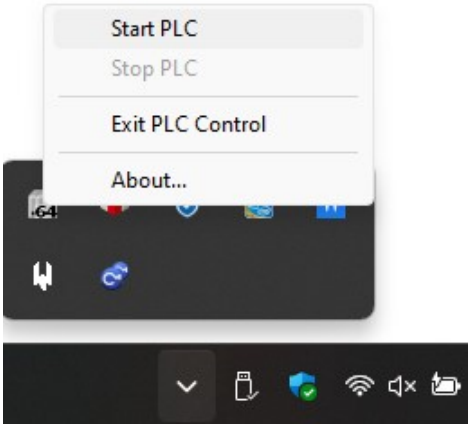


Les éléments de langage à contact sont disponibles dans le volet de droite **Outils**, et dans la barre en haut sous la barre du menu.
Posez un **Contact** à l'aide d'un cliquer-glisser depuis le volet de droite vers le début du premier réseau, comme dans l'image ci-dessus. Le contact apparaît avec des **???** en guise de nom. Saisissez le nom de la variable du contact **BP_Pieton1**, validez avec la touche **Entrée**. Une fenêtre apparaît, confirmez le type booléen de la variable.
Placez à droite du contact un **Bobinage**, c'est-à-dire une sortie. Nommez la **Feu1Rouge**.

Remarquez la liste des variables au-dessus des réseaux. Elle dispose de deux modes d'affichage : tabulaire ou textuel.

Maintenant que votre premier réseau est tracé, vous allez tester votre code.

Dans la barre des tâches Windows, parmi les icônes cachées, cliquez sur **.64** puis sur **Start PLC** afin de démarrer l'automate simulé par le PC, selon l'image ci-contre.

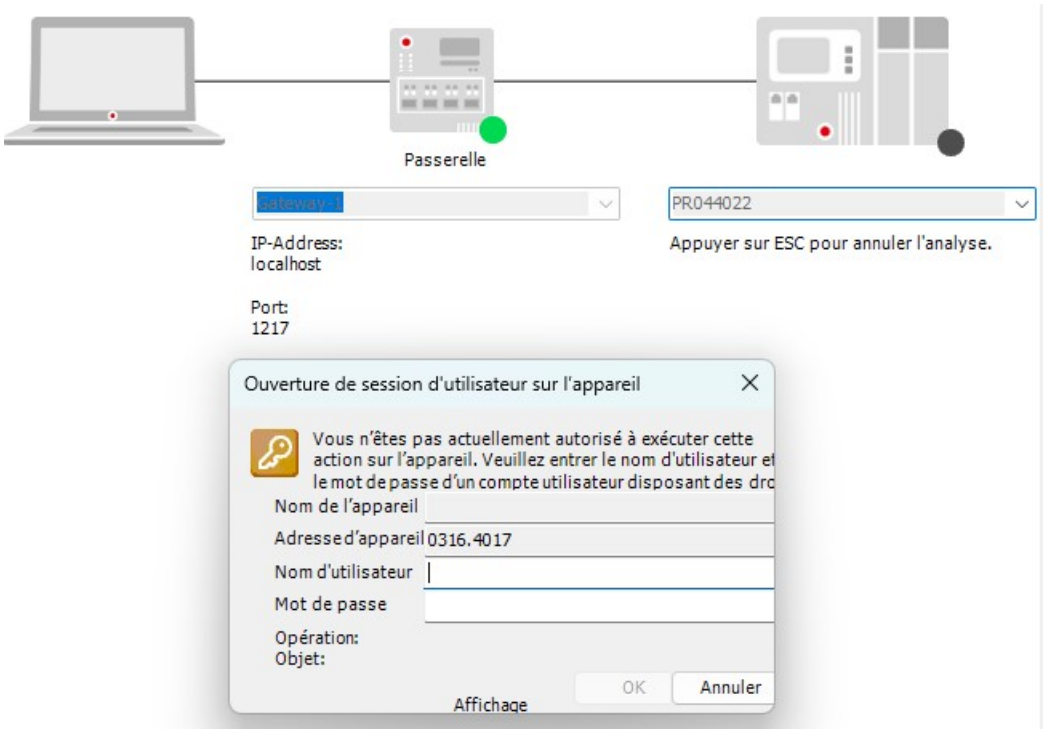


Il est possible qu'un programme soit déjà dans l'automate et génère une fenêtre de visualisation plein écran. Si c'est le cas tapez sur la touche Windows puis revenez à la fenêtre Codesys.



Pour télécharger votre code dans l'automate, dans la barre de menu ci-contre illustrée, cliquez sur **Se connecter**.

Dans la fenêtre qui apparaît, confirmez votre souhait de définir le chemin d'accès à l'automate.
Une fenêtre **Device** comme ci-dessous apparaît alors.
L'accès est protégé par le nom **utilisateur** et le mot de passe **mdp**.
Au final la passerelle et l'automate de la fenêtre ci-dessous doivent passer au vert.



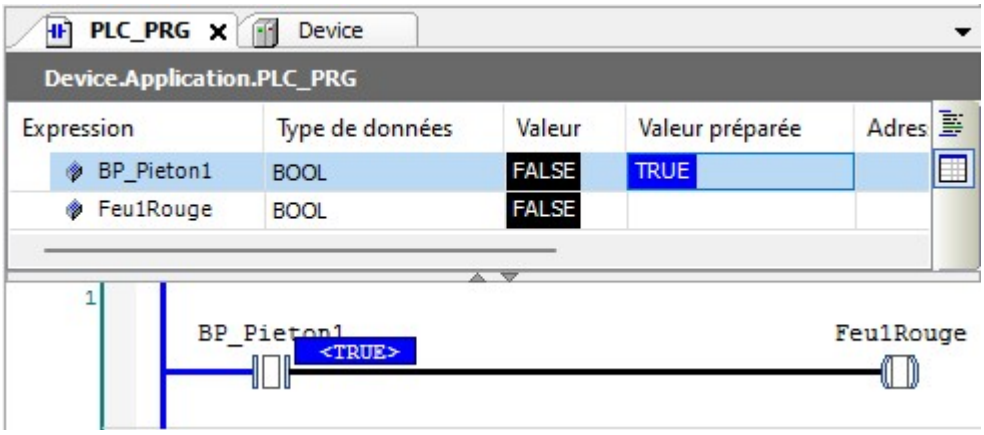
Vous serez parvenu à accéder à l'automate quand l'icône **Se connecter** sera grisée.
Pour y parvenir il est probable que vous deviez cliquer à nouveau sur **Se connecter**, si par exemple l'automate n'est pas démarré ou l'accès pas déverrouillé, etc. N'hésitez pas à écraser le programme déjà présent dans l'automate.

Quand l'icône **Se connecter** est grisée, votre programme est téléchargé dans l'automate, lancez-le en cliquant sur l'icône **Démarrez** dans la barre ci-contre.

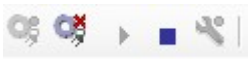


Revenir à la fenêtre **PLC_PRG** afin de tester la programmation.

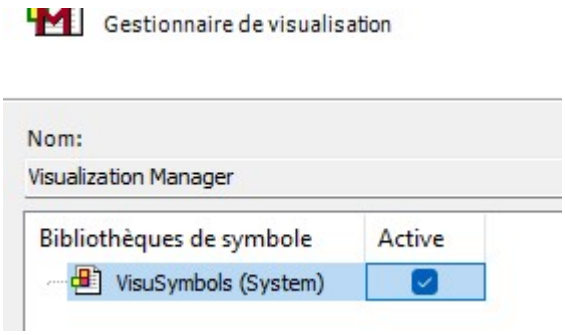
Dans la liste des variables cliquez sur le champ **Valeur préparée** du **BP_Pieton1** afin de la passer à **TRUE** comme sur l'image ci-dessous. Tapez ensuite sur la touche F7 pour forcer cette valeur : le bouton passe à **TRUE** et par conséquent le feu aussi.



Pour tester plus facilement votre code vous allez créer une visualisation.
Commencez par déconnecter l'automate dans la barre ci-contre.



Puis dans le volet de gauche **Appareils** faites un clic droit sur **Application**, allez dans **Ajouter un objet**, **Gestionnaire de visualisation**. La fenêtre ci-contre apparaît, cochez la case pour importer la bibliothèque de symboles et terminez par **Ajouter**. Le chargement est un peu long.

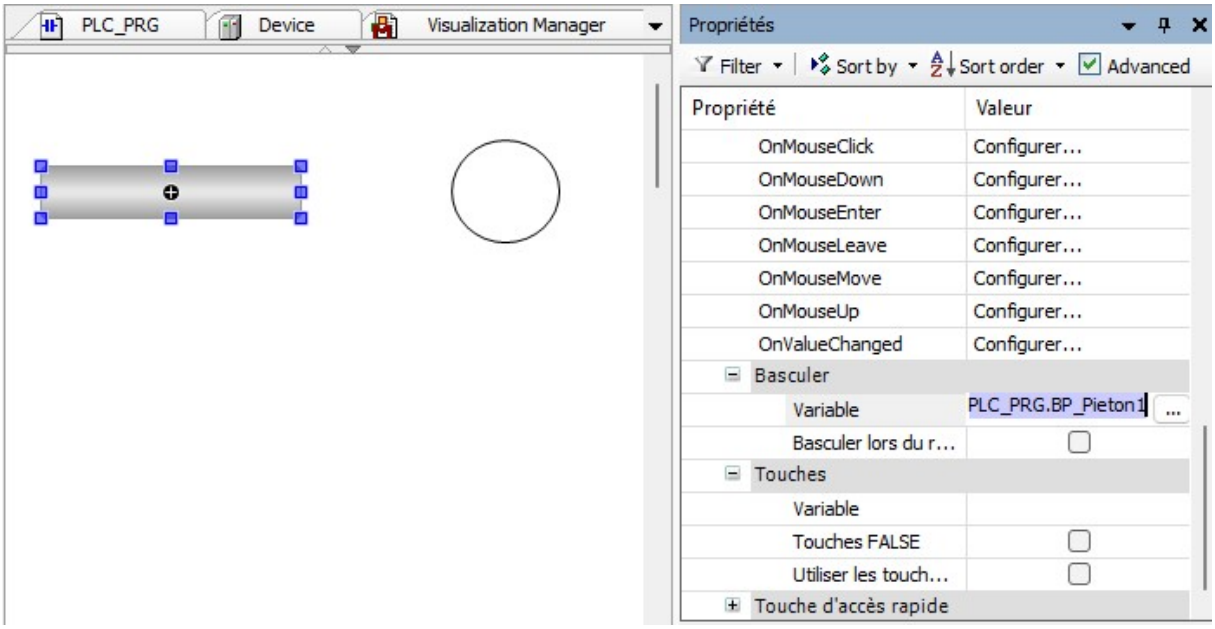


Ensuite faîtes à nouveau un clic droit sur **Application**, allez dans **Ajouter un objet**, **Visualisation**, **Ajouter**.

Vous dessinerez la visualisation dans la fenêtre centrale, à l'aide d'éléments pris dans le volet droit.

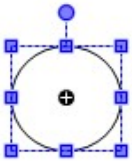
Dans ce volet de droite, dans l'onglet **Boites à outils de visualisation**, dans **Eléments de commande généraux** prenez un **Bouton** et glissez le dans la page de visualisation au centre de l'écran, puis dans **Base** prenez une **Ellipse** et dessinez un cercle pour le feu rouge.

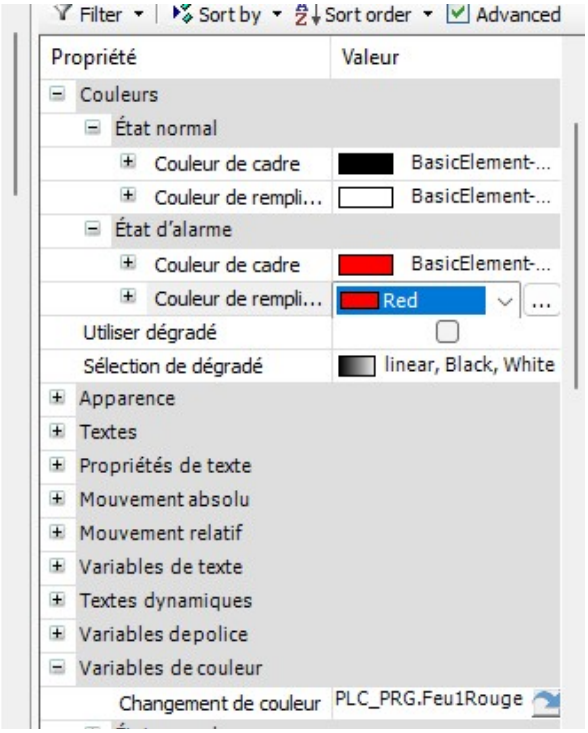
Quand ces deux objets sont dessinés, cliquez sur le dessin du bouton. Ses propriétés apparaissent dans le volet de droite selon l'image ci-dessous.
Sous **Configuration d'entrée**, **Basculer**, **Variable**, cliquez jusqu'à faire apparaître les trois points dans un rectangle comme sur l'image ci-dessous. Cliquez sur les trois points pour ouvrir la fenêtre d'**Aide à la saisie**, allez dans **Application**, **PLC_PRG**, double-cliquez sur la variable **BP_Pieton1**.



Ensuite cliquez sur le dessin du feu pour faire apparaître ses propriétés dans le volet de droite selon l'image ci-dessous. Dans **Couleurs**, **Etat d'alarme**, **Couleur de remplissage**, choisissez le rouge. Dans **Variable de couleur**, choisissez **Feu1Rouge**.







Connectez vous à l'automate pour tester votre programmation. Si une visualisation déjà présente dans l'automate apparaît, cliquez sur la touche Windows pour revenir à Codesys et écraser l'ancien programme.

Remarquez que notre interrupteur à bascule a un comportement différent d'un bouton poussoir.

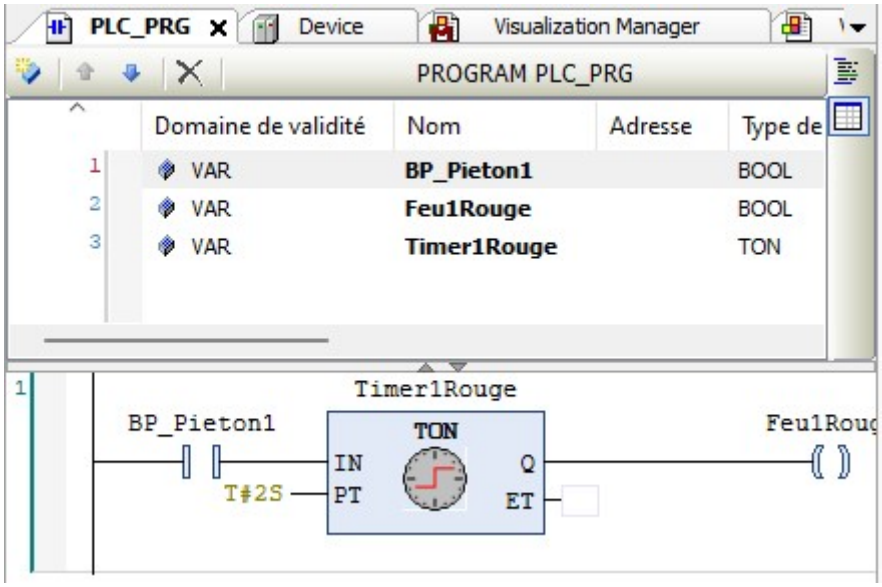
2.2 Ajouter une temporisation de type TON en langage ladder

Déconnectez l'automate, puis dans le programme de l'automate **PLC_PRG**, cliquez entre le bouton et le feu selon l'image ci-dessous, puis cliquez dans la barre d'outils  sur l'icône **Insérer l'appel de module**



Dans la fenêtre qui apparaît, dans **Standard**, **Timer** choisissez une temporisation à l'enclenchement **TON**.

En vous aidant de l'image suivante, nommez la temporisation **Timer1Rouge**, choisissez un **Preset Time** de deux secondes et effacez la variable de la sortie **Elapse Time**.



Testez votre programme avec la visualisation, le feu doit s'allumer deux secondes après le basculement du bouton.

Vous pouvez aussi tester votre code dans la fenêtre ci-dessus : forcez un état **TRUE** en double-cliquant sur **BP_Pieton1** puis en tapant sur la touche F7.
Après que la temporisation est testée, pour supprimez le forçage double-cliquez sur **BP_Pieton1**, réinitialisez l'état du bouton et tapez sur F7.

2.3 Un bouton à bascule et un feu, en programmation structurée

Le langage ST, structured text, est apprécié des techniciens travaillant chez les intégrateurs, c'est-à-dire les entreprises spécialisées en automatisme.

Les entreprises de production sous-traitent aux intégrateurs les études et les installations des chaines de production automatisées, car ce sont des chantiers trop vastes pour leurs services de maintenance.

On peut imaginer une programmation automate avec des parties de code accessibles à la maintenance en ladder et des parties plus sensibles en ST (structured text) accessibles uniquement à l'intégrateur.

La répartition des tâches dépend en particulier des moyens humains dont dispose en interne l'entreprise de production, c'est-à-dire de la taille de son service maintenance.

Vous allez copier-coller certaines parties du précédent projet donc lancez une seconde session de **Codesys V3.5 SP19 Patch3**. Dans cette seconde session, dans le menu, cliquez sur **Fichier**, **Nouveau projet**. Nommez ce second projet **BPFeuTimerST**, restez dans le même dossier **BPFeuTimer**, et choisissez le lange **Littéral Structuré (ST)**. Ouvrez si nécessaire la fenêtre de programmation en double cliquant dans le volet **Appareils** sur **PLC_PRG(PRG)**. Copiez-collez les variables du projet LD vers le projet ST.

Dans le projet ST, dans le volet de gauche **Appareils** faites un clic droit sur **Application**, allez dans **Ajouter un objet**, **Gestionnaire de visualisation**. Dans la fenêtre qui apparaît, cochez la case pour importer la bibliothèque de symbole et terminez par **Ajouter**. Le chargement est un peu long.

Ensuite vous pouvez copier la visualisation du projet LD depuis son volet **Appareils**, puis dans le projet ST, cliquez dans le volet **Appareils** sur **Application** puis clic droit **Coller**.
Vous avez ainsi copié les variables et la visualisation d'un projet à l'autre.
Vous pouvez maintenant fermer le premier projet.

Dans l'onglet PLC_PRG du projet ST vous allez écrire le code suivant :

```
Feu1Rouge := BP_Pieton1 ;
```

Téléchargez votre programme dans l'automate pour le tester.
L'automate lance la fenêtre de visualisation.
La visualisation peut aussi être observée directement dans Codesys dans la page **Visualisation**. Cela peut être utile si vous avez un doute sur l'origine de la fenêtre de visualisation lancée par l'automate. En effet il peut s'agir du code téléchargé par un précédent utilisateur. Par contre dans Codesys, vous êtes sûr qu'il s'agit de votre projet.
Pour ce faire, hors connexion, ouvrez cette page en double-cliquant sur **Visualisation** dans le volet **Appareils**. Connectez-vous à l'automate, celui-ci ouvre la fenêtre de visualisation. Revenez à Codesys à l'aide de la touche Windows afin d'observer et d'utiliser la page de visualisation de Codesys.

2.4 Ajouter une temporisation de type TON en langage ST

Modifiez le code comme ci-après et testez-le. Remarquez la mise à jour du **Timer** en début du cycle de l'automate.

```
// Le timer
Timer1Rouge(PT:=T#2S);

Timer1Rouge.IN:=BP_Pieton1 ;
Feu1Rouge := Timer1Rouge.Q ;
```

2.5 Un bouton poussoir, un feu et une temporisation en langage SFC

Le langage SFC, sequential function chartstructured text, est dérivé du grafcet.

Vous allez copier-coller certaines parties du projet précédent, donc lancez une seconde instance de **Codesys V3.5 SP19 Patch3**. Dans cette seconde fenêtre Codesys, cliquez dans le menu sur **Fichier**, **Nouveau projet**. Nommez le **BPFeuTimerSFC**, restez dans le même dossier **BPFeuTimer**, et choisissez le langage **Diagramme Fonctionnel en Séquence(SFC)**. Si nécessaire double-cliquez sur **PLC_PRG(PRG)** dans le volet **Appareils** pour faire apparaître la fenêtre centrale destinée aux variables et à la programmation. Copiez collez les variables du projet ST vers le projet SFC.

Dans le projet SFC, dans le volet **Appareils** faites un clic droit sur **Application**, allez dans **Ajouter un objet**, **Gestionnaire de visualisation**. N'oubliez pas de cochez la case pour importer la bibliothèque de symboles.
Ensuite vous pouvez copier-coller la visualisation du projet ST vers le projet SFC.

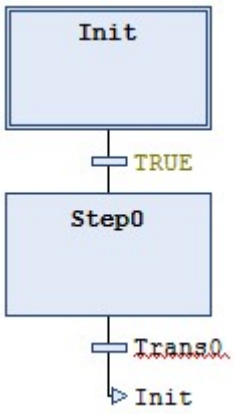
Vous pouvez maintenant fermer le premier projet.

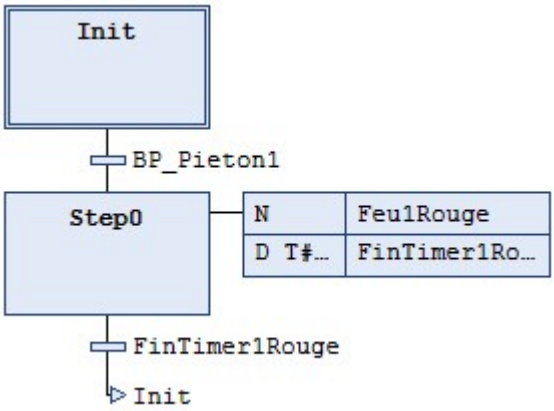
Dans l'onglet **PLC_PRG** du projet SFC vous allez écrire le code sous la forme d'un grafcet.

Tout en bas du grafcet initial, faîtes un clic droit sur la flèche/**Init**, et choisissez **Insérer une transition d'étape**. Vous obtenez le grafcet ci-dessous à gauche.

En vous inspirant de la figure ci-dessous à droite, placez une **Action** à l'étape **Step0** à l'aide d'un cliquer-glisser depuis le volet de droite **Outils**. Puis en utilisant l'**Aide à la saisie** choisissez l'action **Feu1Rouge**. Placez une seconde action sous la première. Saisissez son nom **FinTimer1Rouge**.

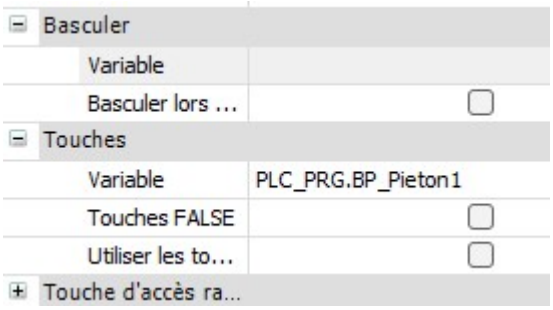
La lettre **N** précédant la première action est son qualificatif, il indique que l'action est activée immédiatement. Changez le qualificatif de la seconde action de **N** à **D** et ajoutez après un espace **T#2S**. Ainsi la seconde action sera activée après un délai de deux secondes. Modifiez les deux transitions selon la figure de droite. Essayez d'en comprendre le principe.





Vous pouvez tester ce programme. L'idée est que le feu rouge s'éteigne après deux secondes. Mais remarquez que l'interrupteur à bascule pose problème. Pour observer la temporisation il faut actionner l'interrupteur une seconde fois pour le placer dans son état initial sous peine de voir la transition **BP_Pieton1** toujours vraie.

Dans la visualisation vous allez donc changer les propriétés du bouton. Retirez la variable **PLC_PRG.BP_Pieton1** de **Basculer** et mettez-la dans **Touches** selon la figure ci-contre. Ainsi vous disposez d'un bouton poussoir et non plus d'un interrupteur à bascule. Vérifiez le comportement du programme.



3. Un feu tricolore en langage ST, LD et SFC

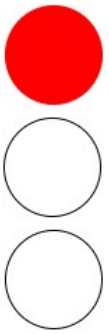
3.1 Un feu tricolore en langage ST

Vous allez commencer en créant un projet à partir d'un projet antérieur écrit en ST, pour cela ouvrez le projet **BPFeuTimerST** créé partie 2.3, allez dans le menu **Fichier**, **Enregistrer le projet sous…** , créez un dossier **UnFeu** au même niveau que le dossier **BPFeuTimer**, y sauvegardez le projet ainsi créé sous le nom **UnFeuST**.

Vous pouvez maintenant modifier le programme ST.
Le but est de coder un feu tricolore unique faisant des cycles.
Inspirez-vous du code CircuitPython en langage structuré partie 2.4 de l'énoncé précédent, en particulier respectez scrupuleusement le découpage du code en parties successives :

```
# Mise à jour des timers
# Les conditions d'évolution
# Les étapes
# Les actions
```

Vous utiliserez trois variables booléennes X0, X10, X20 pour les étapes.
Pour que le feu démarre au rouge choisissez **Feu1Rouge** dans les actions de l'étape X0.
Pour que l'automate démarre dans l'étape **X0**, dans le tableau des variables initialisez X0 à 1.



Les conditions d'évolution, par exemple CE10_20, seront aussi des variables booléennes.

Utilisez les durées suivantes pour les feux vert, orange et rouge :
5 secondes, 1 seconde, 8 secondes.

Vous modifierez la visualisation pour avoir un feu tricolore.

3.2 Un feu tricolore en langage LD

Sans fermer le projet de la page précédente, dans une seconde instance Codesys ouvrez le projet **BPFeuTimerLD** de la partie 2.2, allez dans le menu **Fichier**, **Enregistrer le projet sous…** , sauvegardez dans le dossier **UnFeu** le projet ainsi créé sous le nom **UnFeuLD**.

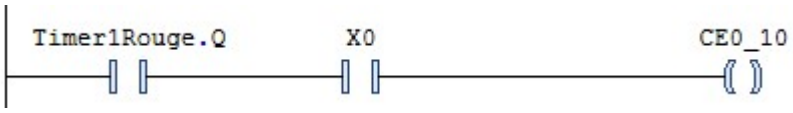
Vous allez modifier la programmation pour obtenir un feu tricolore unique.

Vous allez d'abord récupérer une partie du travail du projet de la page précédente.
Commencez par copier-coller les variables qui vous manquent depuis le projet **UnFeuST** vers le nouveau projet **UnFeuLD**.
Ensuite remplacez la visualisation actuelle par celle de **UnFeuST**. Vous pouvez alors fermer le projet ST.

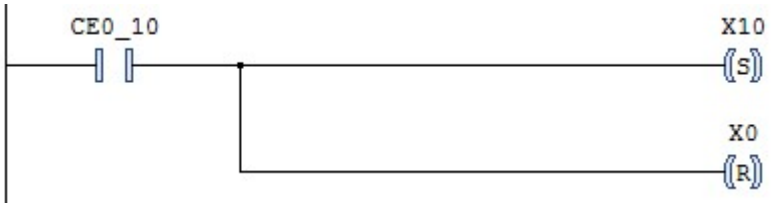
Vous allez enfin créer les réseaux nécessaires à la programmation d'un feu tricolore en respectant toujours le découpage en trois parties successives :

```
# Conditions d'évolution
# Etapes
# Actions
```

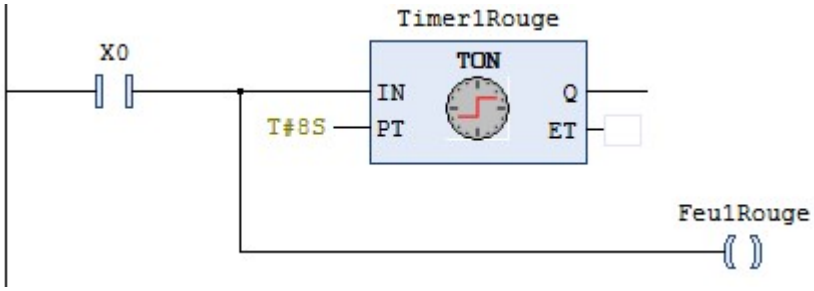
Le réseau suivant est un exemple d'écriture d'une condition d'évolution.



Ce second réseau est un exemple d'écriture d'évolution des étapes. Utilisez les outils **Définir bobinage** et **Réinitialiser bobinage** du volet droit ou les outils **Insérer bobinage Set** et **Insérer bobinage Reset** de la barre d'outils située sous la barre du menu de Codesys.



Ce dernier réseau ci-dessous illustre les actions. Remarquez que ce réseau intègre aussi la mise à jour de la temporisation.



Testez votre code.
Pendant que l'automate tourne, revenez à la page du programme en ladder et observez l'évolution des variables.

3.3 Un feu tricolore en langage SFC

Dans une seconde instance de Codesys, ouvrez le projet **BPFeuTimerSFC** de la partie 2.5, allez dans le menu **Fichier**, **Enregistrer le projet sous…** , sauvegardez dans le dossier **UnFeu** le projet ainsi créé sous le nom **UnFeuSFC**.

Vous allez modifier la programmation pour obtenir un feu tricolore unique.

Vous allez d'abord récupérer une partie du travail de la page précédente.
Commencez par copier-coller les variables qui vous manquent et remplacez la visualisation actuelle par celle de **UnFeuLD**.
En SFC, les étapes et les conditions d'évolution (transitions) sont implicites et ne doivent donc pas apparaitre dans le tableau des variables. Effacez-les si nécessaire.
Rappelez-vous que notre façon de coder en LD et en ST découlait d'un cahier des charges sous forme d'un grafcet, qui nécessitait de créer ces variables.

Modifiez la programmation de type grafcet pour coder le fonctionnement du feu tricolore. Nommez les étapes X0, X10 et X20.
La programmation SFC intègre une étape initiale, la première, dont le cadre est doublé. Il n'est donc pas nécessaire d'initialiser X0 à 1 en SFC.

Testez votre code.

Pendant que l'automate tourne, revenez à la page de la programmation en grafcet et observez l'évolution des variables et des étapes.

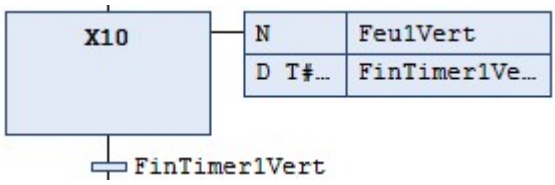
4. Carrefour de feux tricolores en langage SFC

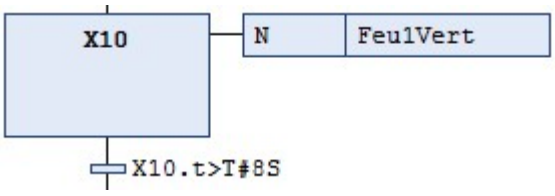
4.1 Deux feux tricolores en langage SFC

A partir du précédent projet **UnFeuSFC**, allez dans le menu **Fichier**, **Enregistrer le projet sous…** , sauvegardez dans un nouveau dossier **DeuxFeux** le projet ainsi créé sous le nom **DeuxFeuxSFC**.

En vous aidant du code en CircuitPython, partie 4.1 de l'énoncé précédent, vous allez modifier la programmation de type grafcet pour coder le fonctionnement de deux feux tricolores.

Vous pouvez commencer, pour alléger la programmation, par modifier la façon de coder les temporisations, en vous inspirant des deux grafcets ci-dessous qui ont le même comportement.
Dans le premier cas, l'action **FinTimer1Vert** est activée après un délai, et est utilisée comme transition.
Dans le second cas, une condition sur la durée de l'étape est utilisée comme transition.
Changez pour la seconde façon de programmer.





Le codage du feu tricolore nécessite six étapes X0 à X50.

Pendant une seconde les feux rouges des deux axes sont allumés en même temps
Utilisez les durées suivantes pour les feux vert et orange : 5 secondes et 1 seconde.

Ajoutez le feu tricolore de l'axe secondaire à la visualisation.

Testez votre code.

Vous pouvez enrichir la visualisation en vous inspirant de la page 2 du précédent énoncé.

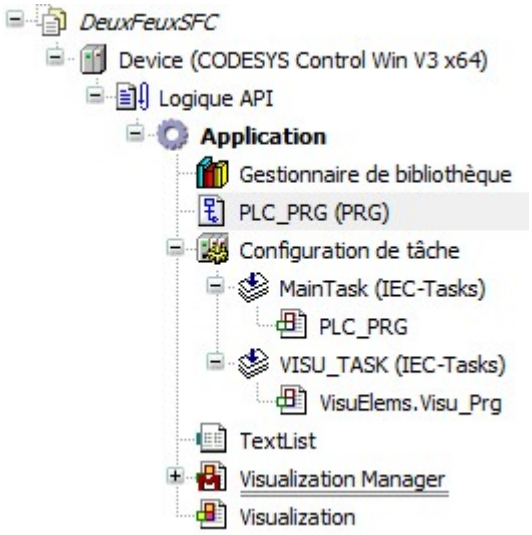
4.2 Carrefour de feux tricolores avec mode Jour/Nuit en langage SFC

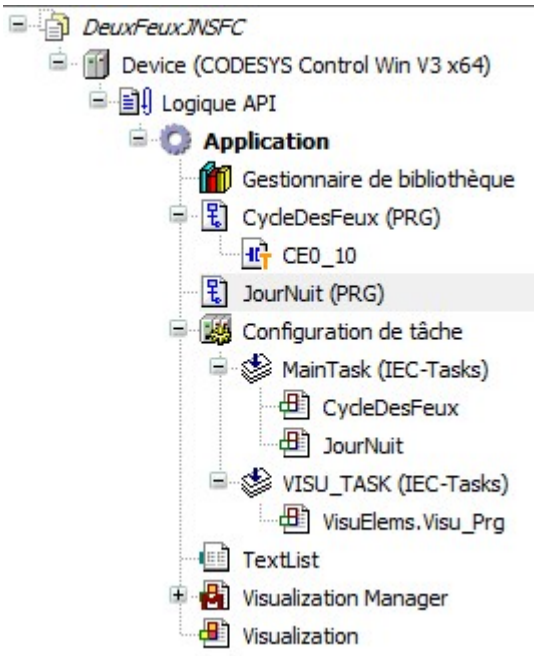
A partir du précédent projet **DeuxFeuxSFC**, allez dans le menu **Fichier**, **Enregistrer le projet sous…** , sauvegardez le projet ainsi créé sous le nom **DeuxFeuxJNSFC**.

Les modifications du volet **Appareils** du projet que vous devrez apporter sont illustrées ci-dessous.

Vous allez ajouter un second grafcet pour le mode Jour/Nuit.
Commencez par renommer le programme SFC du cycle des feux : dans le volet **Appareils**, faites un clic droit sur **PLC_PRG(PRG)** cliquez sur **Refactoring**, puis **Renommer PLC_PRG…**, saisissez le nouveau nom **CycleDesFeux**. Acceptez que Codesys modifie toutes les occurrences du nom du programme.

Ensuite ajoutez un programme : clic droit sur **Application**, **Ajouter un objet**, **POU…**
Une POU, Program Organization Units, est une unité de programmation (programme, fonction, bloc fonction). Donnez-lui le nom **JourNuit**, choisissez le type **Programme** et le langage SFC.

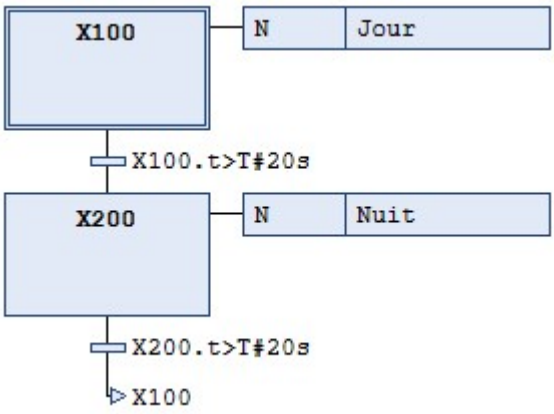




Pour éditer le second grafcet, double-cliquez dans le volet **Appareils** sur **JourNuit(PRG)** et saisissez le diagramme ci-contre. Les deux temporisations de 20 secondes simulent l'alternance du jour et de la nuit.

Pour faciliter le débogage, dans la visualisation ajoutez un rectangle qui change de couleur avec la variable **Jour**.

Vous pouvez déjà tester la programmation. Vous constaterez que seul le cycle des feux tourne.



Pour y remédier, dans le volet **Appareils** double-cliquez sur **MainTask**. Puis dans la fenêtre de la tâche principal cliquez sur **Ajouter l'appel**, dans **Application** double cliquez sur **JourNuit**. Vérifiez que maintenant les deux POU sont animées par l'automate.

Modifiez votre programmation afin que le jour les feux fassent des cycles normaux en boucle et que la nuit le feu de l'axe principal reste toujours au vert.

Enfin modifiez la programmation pour que, la nuit, la présence d'un véhicule sur l'axe secondaire ou l'appui sur un des deux boutons piétons provoque un cycle unique des feux.
Pour cela ajoutez dans la visualisation les deux boutons ainsi qu'un **Polygone** de la forme d'un parallélogramme symbolisant la boucle magnétique sous l'enrobé.

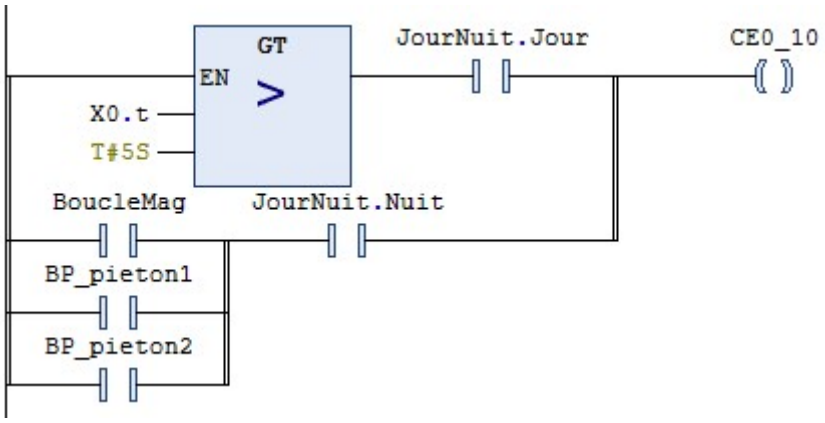
4.3 Carrefour de feux tricolores avec mode Jour/Nuit en langage SFC Transition en ladder - Texte jour/nuit dynamique

La condition d'évolution CE0_10 se présente sous la forme d'une ligne logique un peu longue et assez peu lisible. On se propose de l'écrire en ladder.
Faites un clic droit sur **CycleDesFeux(PRG)**, **Ajouter un objet**, **Transition**. Donnez-lui le nom CE0_10 et choisissez le langage LD.

Ecrivez la condition en ladder selon l'illustration ci-dessous.
Les interrupteurs en série forment des ET logiques.
En parallèle ils forment des OU.

Prenez le bloc de **Comparaison supérieur à** dans le volet **Outils** dans les **Opérateurs mathématiques**.

Pour tracez un contact en parallèle, utilisez dans le volet **Outils**, dans les **Eléments de langage à contact**, l'outil **Contact parallèle** ou **Début/Fin de branchement de ligne**.



Enfin dans le grafcet du cycle des feux, au niveau de la transition 0_10, utilisez l'aide à la saisie, dans la catégorie **Sous-objet**, choisissez **CE0_10**.

Dans la visualisation, le rectangle qui change de couleur selon la période jour ou nuit est trop peu explicite. Vous allez ajouter un texte qui change selon la période. Trouvez sur Internet une solution pour que le texte **Jour** ou le texte **Nuit** apparaisse dans ce rectangle.

4.4 Réglage des durées des feux dans la visualisation

Ajoutez dans la visualisation six **Spin Box**, une par durée à régler. Dans les propriétés de chaque Spin Box, dans **Position**, indiquez le nom de la variable à affecter.



© Earthz Stocker

Créez une POU en ST pour convertir les entiers des Spin Box en temps. Vous pouvez utiliser l'exemple de code suivant.

```
Tempo1Vert := 1000 * INT_TO_TIME(T1Vert);
```

Utilisez ces variables converties en temps dans les transitions du cycle des feux.

Dans le volet **Appareils**, n'oubliez pas de glisser la nouvelle POU dans la tâche principale.

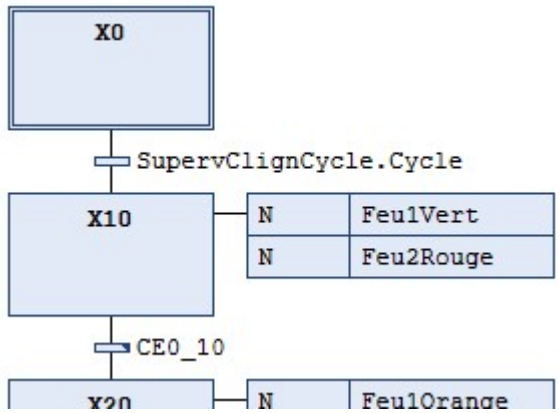
4.5 Clignotement des feux orange

A la mise sous tension, et lors d'une panne (cas traité plus tard), les feux orange doivent clignoter.
La mise en route du cycle des feux doit se faire en présence d'un technicien de maintenance.

Le projet va comporter 4 grafcets.

Celui du cycle jour nuit est inchangé.

Celui du cycle des feux est modifié, selon l'exemple suivant afin d'intégrer une étape supplémentaire. X0 est une étape initiale d'attente sans action, pendant laquelle le grafcet des feux clignotants (à créer) tourne. Et vice-versa, quand le cycle normal des feux tourne, celui des feux clignotants orange est en attente.



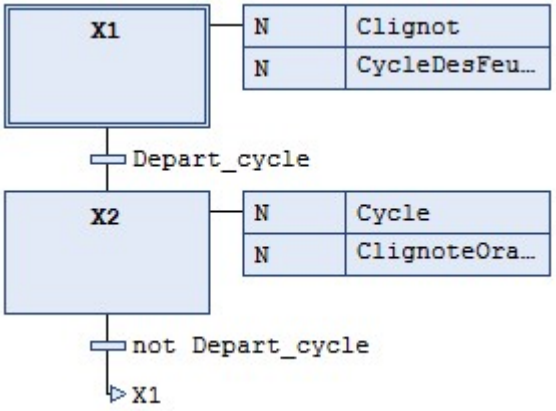
Le grafcet du cycle de clignotement des feux orange devra être créé et devra donc aussi intégrer une étape initiale d'attente. Utilisez une durée des feux orange de 400 ms.

Enfin un grafcet superviseur doit être créé, selon l'exemple ci-dessous.

Depart_cycle est un interrupteur à bascule qu'il vous faudra ajouter à la visualisation. Il sera actionné par le technicien de maintenance.

Dans l'étape initiale, l'action **Clignot** activera le cycle des feux orange clignotants. La seconde action **CycleDesFeux.SFCInit** initialisera le grafcet du cycle des feux tricolores.

Dans la seconde étape, l'action **Cycle** active le cycle des feux tricolores comme on peut le voir dans la première transition du grafcet de la page précédente. La seconde action **ClignoteOrange.SFCInit** initialisera le grafcet du cycle des feux orange clignotants.



Pour que cela fonctionne il faut d'abord autoriser l'usage des deux **indicateurs SFC** (ou flag SFC) d'initialisation.
Depuis le volet **Appareils**, clic droit sur **CycleDesFeux(PRG**), **Propriétés**, **Paramètres SFC**, décochez **Utiliser les valeurs par défaut**, et cochez **SFCInit**.
Faites de même dans le cycle de clignotement des feux orange.

Vous pouvez maintenant utiliser **CycleDesFeux.SFCInit** et **ClignoteOrange.SFCInit** dans le grafcet superviseur.

Le fonctionnement des feux risque d'être erratique, en particulier celui des feux orange. En effet, deux grafcets commandent les mêmes sorties : les feux orange.

Vous allez donc créer une POU qui élabore les feux orange utilisés dans la visualisation à partir des variables des feux orange issues des deux POU **CycleDesFeux** et **ClignoteOrange**.

4.6 Détection de la panne

Pour une question de sécurité, les feux rouges sont indispensables. Donc si l'un d'entre-eux est défectueux, les feux orange doivent clignoter.

Trouvez une idée pour simuler la panne d'un feu rouge en cliquant dessus dans la visualisation. Utilisez une variable **Panne** dans le grafcet superviseur pour répondre au cahier des charges.